

Centro Universitario de Ciencias Exactas e Ingenierías



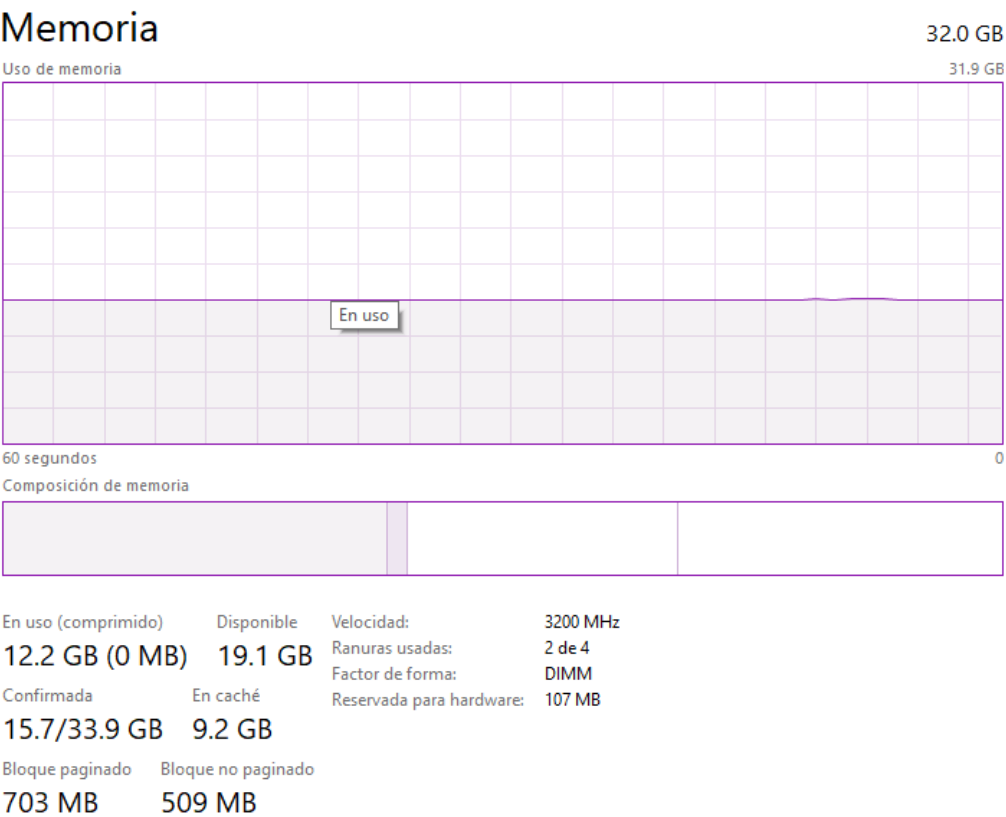
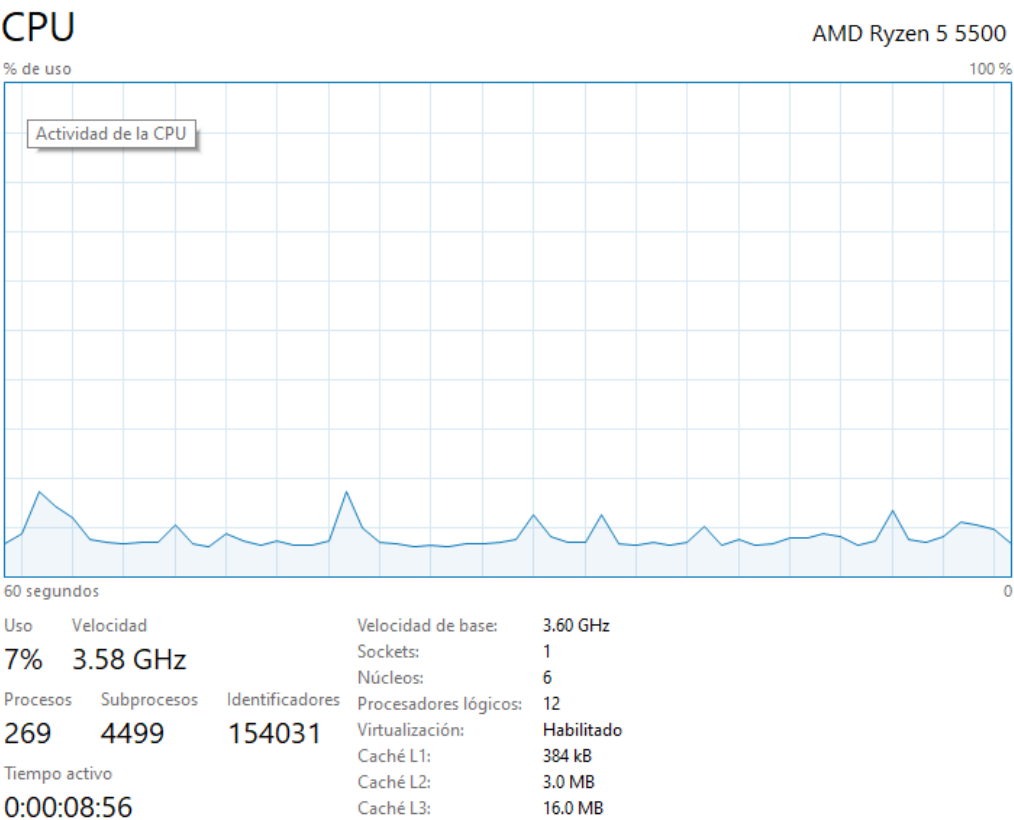
Programación Paralela y Concurrente - D02

Proyecto Final

Víctor Alejandro Hernández Briseño

Especificaciones del Hardware de Prueba:

Para la realización de este programa se utilizo de CPU un AMD Ryzen 5 5500, con caché L1 de 384kB, L2 3MB y L3 16MB. además de contar con 32GB de RAM



Sección de Metodología:

De lado de la IA fue utilizada chatgpt con los siguientes prompts:

- Genera un programa completo en C++ puramente secuencial que realice las siguientes tareas:

1. Generar una imagen 8K del conjunto de Mandelbrot.
2. Guardar la imagen en formato PPM.
3. Aplicar un filtro de convolución 2D pesado utilizando un desenfoque Gaussiano.
4. Guardar la imagen filtrada.
5. Medir el tiempo de ejecución de ambas tareas usando chrono.
6. Utilizar únicamente bibliotecas estándar de C++.

- Paraliza el siguiente programa secuencial en C++ utilizando OpenMP.

1. Paralelizar la generación del conjunto de Mandelbrot.
2. Paralelizar el filtro Gaussiano.
3. Utilizar distintos schedulers de OpenMP:
 - static
 - dynamic
 - guided
4. Permitir modificar el chunk size.
5. Medir tiempos de ejecución para comparar rendimiento.
6. Agregar impresión de progreso en terminal.
7. Mantener compatibilidad con compilación usando g++ y -fopenmp.

- Modifica los bucles internos del filtro de convolución Gaussiana para forzar vectorización SIMD utilizando OpenMP SIMD.

1. Utilizar `#pragma omp simd`.
2. Aplicar reduction en las variables acumuladoras.
3. Mantener compatibilidad con OpenMP.
4. Permitir verificación de vectorización mediante:
 - `-O3`
 - `-march=native`
 - `-fopt-info-vec`

Dentro de algunos problemas y cuellos de botella, el código inicial generado por IA era funcional, pero presentaba limitaciones típicas de aplicaciones HPC no optimizadas:

- ausencia de paralelismo
- falta de balance dinámico
- inexistencia de vectorización SIMD
- sincronización excesiva
- poca optimización de memoria/cache

La incorporación progresiva de:

- OpenMP
- scheduling dinámico
- SIMD
- histogramas privados
- afinidad de hilos

permitió transformar el programa en una aplicación considerablemente más eficiente y escalable para procesamiento paralelo de imágenes 8K.

Gráfica de Tiempo de Ejecución vs. Número de Hilos

Ya dentro de la paralelización con OpenMP, tuvimos los siguientes comportamientos dependiendo el numero de hilos y si estaba paralelizado o no.

En este primer caso, se hizo sin ninguna clase de paralelización.

Notando que tardó bastante tiempo para ambas tareas. Tomando en total 729 segundos.

```
"C:\Users\Ale\Desktop\á\10 DECIMO SEMESTRE WTF\PROGRAMACION CON  
Generando fractal Mandelbrot 8K...  
Fractal: 100% completado  
Imagen fractal guardada.  
  
Aplicando blur Gaussiano...  
Blur: 100% completado  
Imagen blur guardada.  
  
=====
```

Métrica	Valor
Tiempo fractal	368.697 segundos
Tiempo blur	360.777 segundos
Tiempo total	729.611 segundos

```
=====
```

Process returned 0 (0x0) execution time : 729.771 s
Press any key to continue.

Mas tarde con el codigo paralelizado y usando solo 2 hilos, este solo le tomo 228 segundos, menos de la mitad de tiempo que sin paralelizar.

```
=====
```

RESULTADOS FINALES

```
=====
```

Threads usados: 2
Tiempo total: 228.786 segundos

```
=====
```

Asi mismo, usando 4 hilos este redujo el tiempo a tan solo 175 segundos.

```
=====
```

RESULTADOS FINALES

```
=====
```

Threads usados: 4
Tiempo total: 175.418 segundos

```
=====
```

Utilizando únicamente 8 hilos, este solo le tomo 120 segundos en terminar\

RESULTADOS FINALES

Threads usados: 8

Tiempo total: 120.392 segundos

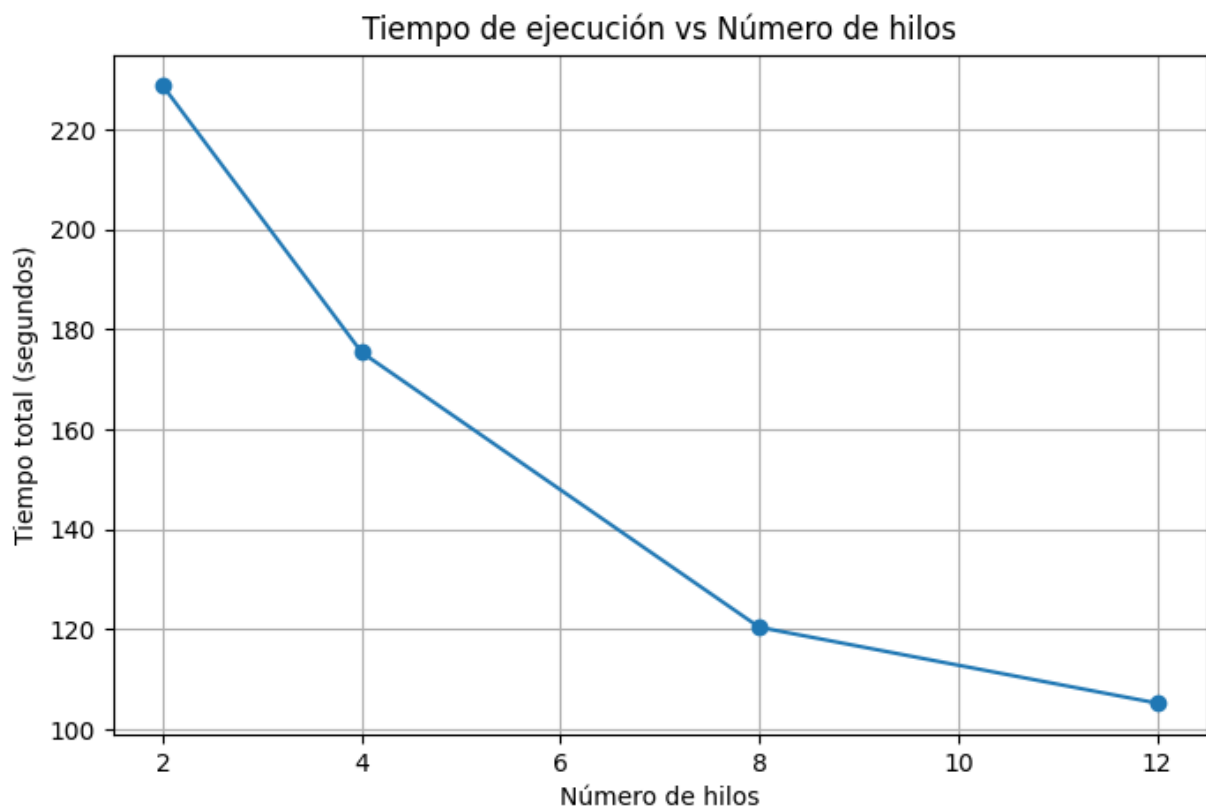
Por último, el máximo de hilos que puede mi procesador que son 12, redujo el tiempo a tan solo 105 segundos.

RESULTADOS FINALES

Threads usados: 12

Tiempo total: 105.214 segundos

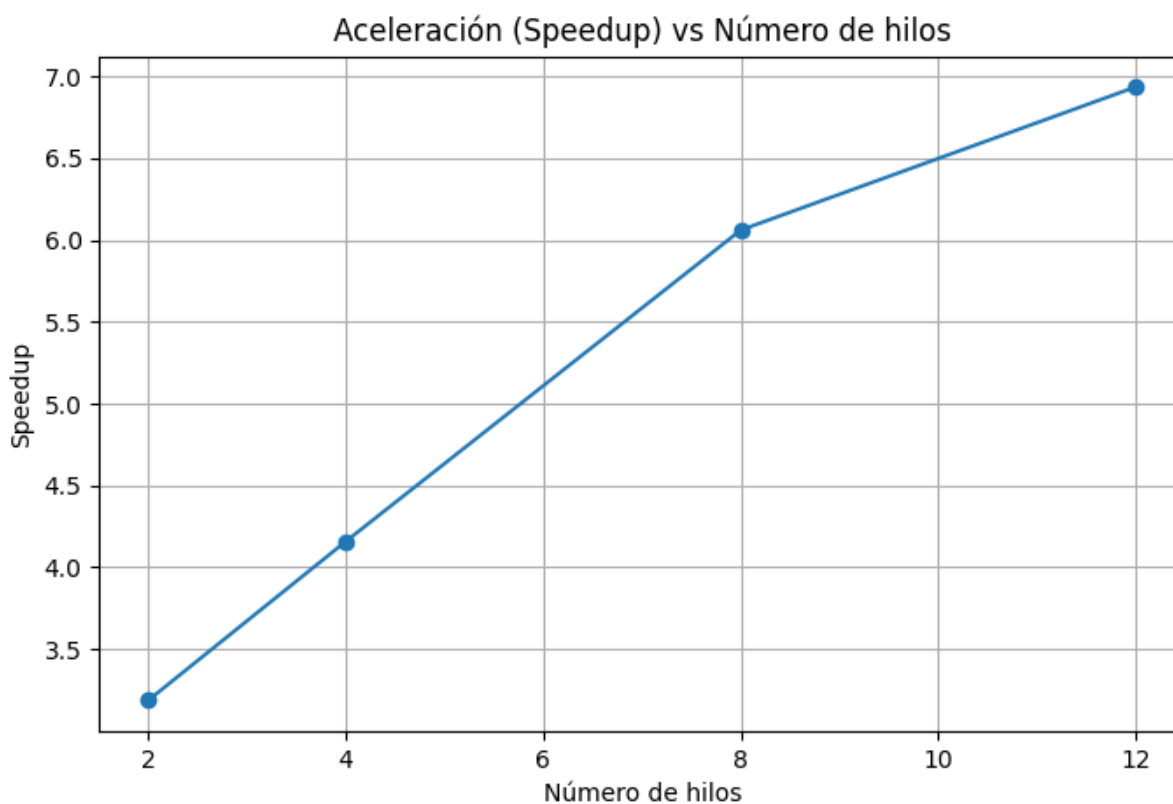
Dándonos esta grafica del tiempo que fue reduciendo con diferente cantidad de hilos usados



Gráfica de Aceleración (Speedup) vs. Número de Hilos.

Ya teniendo los valores de tiempo podemos sacar el Speedup, usando

$$\text{Speedup} = T_{\text{secuencial}} / T_{\text{paralelo}}$$



Análisis de Rendimiento:

Como ya vimos en los resultados de la velocidad dependiendo al número de hilos tenemos:

Threads	Tiempo (s)	Speedup
1 (base no paralelizado)	729.611	1.00
2	228.786	3.19
4	175.418	4.16
8	120.392	6.06
12	105.214	6.93

Podemos notar que de 4 a 8 hay una mejora bastante significativa, a comparación de 8 a 12, ya que solo se reducen apenas 15 segundos usando 4 hilos más.

Según el límite impuesto por la ley de Amdahl, el límite se comienza a observar a partir de los 8 hilos, ya que la ganancia marginal disminuye y la eficiencia paralela cae. Por lo tanto, el overhead comienza a dominar entre los 8 y 12 hilos.

después de paralelizar las tareas usando OpenMP, se evaluó el tiempo que toma en generar la imagen el código, dividiéndose entre static, Dynamic y guided, cada uno con diferentes Chunk size, ya que se debía encontrar el que menor tiempo tardara, así se sabía cual era el más óptimo para el procesador.

Los resultados de dicha evaluación fue la siguiente:

```
=====
GENERACION PARALELA MANDELBROT 8K
OpenMP Threads: 12
=====

Scheduler: static
Chunk Size: 0
=====
Progreso: 100%444%
Tiempo total: 13.9456 segundos
Imagen guardada: mandelbrot_static_0.ppm

=====

Scheduler: dynamic
Chunk Size: 1
=====
Progreso: 100%556%
Tiempo total: 10.1584 segundos
Imagen guardada: mandelbrot_dynamic_1.ppm
```

```
Scheduler: dynamic
Chunk Size: 10
=====
Progreso: 100%222%
Tiempo total: 10.4423 segundos
Imagen guardada: mandelbrot_dynamic_10.ppm

=====

Scheduler: dynamic
Chunk Size: 50
=====
Progreso: 100%481%
Tiempo total: 9.65682 segundos
Imagen guardada: mandelbrot_dynamic_50.ppm

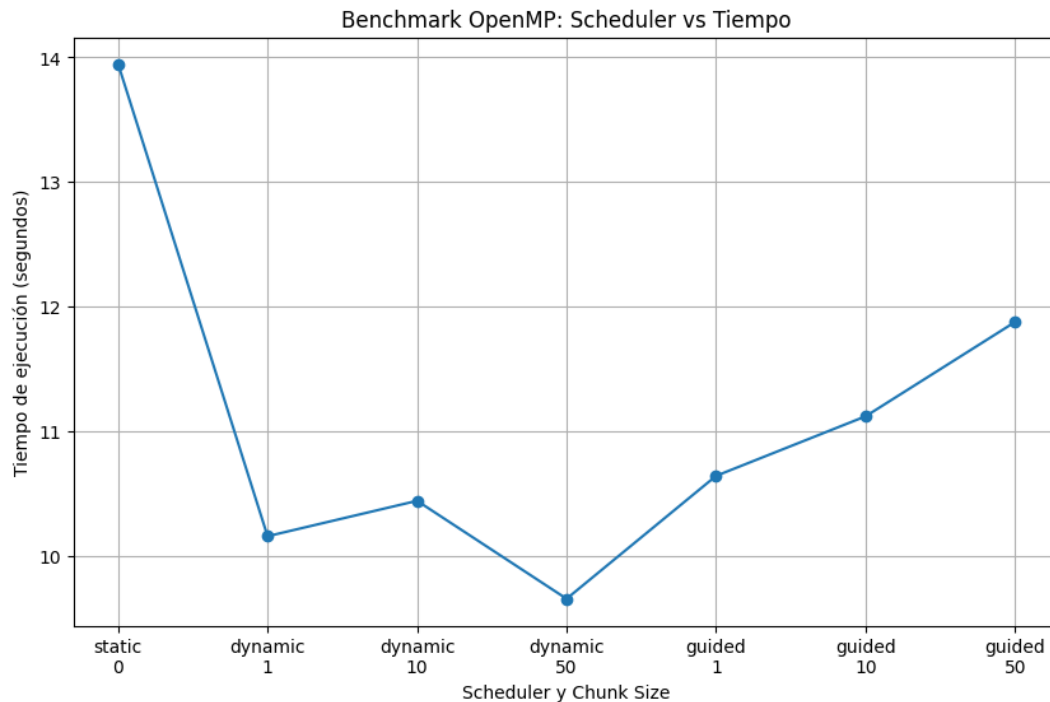
=====

Scheduler: guided
Chunk Size: 1
=====
Progreso: 100%222%
Tiempo total: 10.6423 segundos
Imagen guardada: mandelbrot_guided_1.ppm

=====

Scheduler: guided
Chunk Size: 10
=====
Progreso: 100%889%
Tiempo total: 11.1198 segundos
Imagen guardada: mandelbrot_guided_10.ppm
```


Lo que graficando nos lleva a que el mas optimo para un Ryzen 5 5500, es Dynamic con 50 de chunk size, con apenas 9.6 segundos para realizar la tarea 1



Ahora hablando ya del funcionamiento se tomó en cuenta la siguientes cosas:

La primera forzar la vectorización utilizando la estructura SPMD en el filtro Gaussiano.

```
PS C:\Users\Ale\Desktop\final> g++ -O3 -march=native -fopenmp -fopt-info-vec proyectoF
inal.cpp -o proyectoFinal.exe
proyectoFinal.cpp:227:25: optimized: loop vectorized using 32 byte vectors
proyectoFinal.cpp:393:31: optimized: loop vectorized using 32 byte vectors
proyectoFinal.cpp:393:31: optimized: loop versioned for vectorization because of poss
ible aliasing
proyectoFinal.cpp:115:13: optimized: basic block part vectorized using 32 byte vectors
C:/msys64/mingw64/include/c++/14.2.0/bits/stl_vector.h:99:4: optimized: basic block pa
rt vectorized using 16 byte vectors
C:/msys64/mingw64/include/c++/14.2.0/bits/stl_vector.h:99:4: optimized: basic block pa
rt vectorized using 16 byte vectors
C:/msys64/mingw64/include/c++/14.2.0/bits/basic_ios.h:466:2: optimized: basic block pa
rt vectorized using 32 byte vectors
C:/msys64/mingw64/include/c++/14.2.0/fstream:912:9: optimized: basic block part vector
ized using 16 byte vectors
C:/msys64/mingw64/include/c++/14.2.0/bits/stl_vector.h:99:4: optimized: basic block pa
rt vectorized using 16 byte vectors
C:/msys64/mingw64/include/c++/14.2.0/bits/stl_vector.h:398:25: optimized: basic block
part vectorized using 16 byte vectors
proyectoFinal.cpp:76:27: optimized: loop vectorized using 32 byte vectors
proyectoFinal.cpp:76:27: optimized: loop vectorized using 16 byte vectors
C:/msys64/mingw64/include/c++/14.2.0/bits/stl_vector.h:99:4: optimized: basic block pa
rt vectorized using 16 byte vectors
C:/msys64/mingw64/include/c++/14.2.0/bits/stl_vector.h:398:25: optimized: basic block
part vectorized using 16 byte vectors
proyectoFinal.cpp:208:13: optimized: basic block part vectorized using 16 byte vectors
C:/msys64/mingw64/include/c++/14.2.0/bits/stl_vector.h:99:4: optimized: basic block pa
rt vectorized using 16 byte vectors
C:/msys64/mingw64/include/c++/14.2.0/bits/stl_vector.h:99:4: optimized: basic block pa
rt vectorized using 16 byte vectors
```

Además de que se utilizó variables de entorno OMP_PROC_BIND y OMP_PLACES, aquí se establecieron. En true y cores

```
Microsoft Windows [Versión 10.0.1904
(c) Microsoft Corporation. Todos los
C:\Users\Ale>set OMP_PROC_BIND=true
C:\Users\Ale>set OMP_PLACES=cores
C:\Users\Ale>
```

Con estas especificaciones generamos esto:

```
MANDELBROT 8K HPC
OpenMP Threads: 12
=====
=====
GENERANDO MANDELBROT 8K
Scheduler: dynamic
Chunk size: 50
=====
Fractal: 100%593%
Tiempo fractal: 9.0132 segundos
Imagen fractal guardada.
```

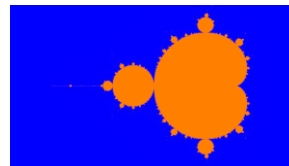
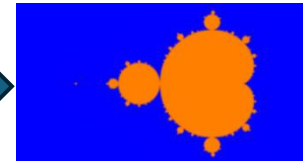


Imagen 8K



Filtro Gaussiano

```
=====
HISTOGRAMA LOCAL PRIVADO
=====
Tiempo histogram local: 0.0146156 segundos

Primeros colores:
Color 0: 15200576
Color 1: 7536722
Color 2: 1188365
Color 3: 412618
Color 4: 270641
Color 5: 189873
Color 6: 132162
Color 7: 104414
Color 8: 73903
Color 9: 54077
```

```
=====
FILTRO GAUSSIANO PARALELO SIMD
Radius: 35
Sigma: 20
=====
Blur: 100%648%
Tiempo blur SIMD: 50.0516 segundos
Imagen blur guardada.

=====
HISTOGRAMA ATOMIC
=====
Tiempo histogram atomic: 0.367187 segundos
```

```
Primeros colores:
Color 0: 15200576
Color 1: 7536722
Color 2: 1188365
Color 3: 412618
Color 4: 270641
Color 5: 189873
Color 6: 132162
Color 7: 104414
Color 8: 73903
Color 9: 54077
```

Finalmente el programa realiza las dos tareas, utilizando el Scheduler Dynamic con 50 de Chunk size para crear la imagen, forzando la vectorización SIMD para el filtro Gaussiano, dando un recuento de los colores utilizados en la imagen y todo paralelizado con 12 hilos usando OpenMP y mejorando el uso del caché L1 y L2 con OMP_PROC_BIND y OMP_PLACES.

Obtenemos un tiempo total de 59 segundos

Conclusiones:

Este proyecto se desarrolló una aplicación de generación de imágenes 8k y convertirlas con filtro gaussiano donde se utilizaron varias técnicas para buscar la mejor optimización y eficiencia, donde se generó la imagen 8K con el conjunto Mandelbrot, se utilizó un scheduler Dynamic con 50 de chunk size el cual se buscó directamente cual era el más óptimo para el procesador, se forzó la vectorización para el filtro gaussiano mediante `#pragma omp simd`, permitiendo al compilador utilizar instrucciones AVX/SSE y mejorando significativamente el throughput matemático del programa. Se usó de exclusión mutua mediante `#pragma omp atomic`, Se usó de histogramas privados por hilo, Se utilizaron variables de entorno para mejorar el uso de la memoria caché, todo paralelizado para usarse con máximo 12 hilos el cual se evaluó con diferentes usos de hilos, para buscar el que diera mayor eficiencia.